
Neurosymbolic Supervision with Explicit Evidence: A Cold Repair Study

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We study whether native source-linked semantic context improves cold program
2 repair under fixed resources. Our supervision design records the subject, scope,
3 and authority of evidence separately from operational completion. We compare
4 matched public raw context (A) with the same raw context plus native semantic
5 context (B), using Grok 4.6 across eight historical repository families and two
6 nested repetitions per arm. All 32 fixed cells reached terminal accounting after
7 one proposal POST each. Nine yielded useful repairs: A achieved 5/16 and B
8 4/16. The mean family-level B-minus-A difference was -0.0625 , with a frozen
9 descriptive 95% percentile interval of $[-0.1875, 0]$. Fourteen requests reached
10 the child deadline. Sixteen scoring invocations included one incomplete hidden-
11 validation import failure. All provider charges remain unsettled. These results offer
12 no observed advantage for the combined semantic-context treatment in this small
13 convenience sample. General semantic correctness, human fidelity, and withdrawn
14 routing, reuse, and publication benefits remain unmeasured.

15 1 Introduction

16 A coding supervisor must distinguish what an edit changes, what evidence applies to the resulting
17 program, and who may accept the transition. A stored passing result can concern an earlier source
18 revision; a completed task can record an operational decision without establishing a behavioral
19 property. Repository-agent interfaces and test-validation benchmarks make the boundary between
20 proposals and checks central to useful repair [6, 3]. Our starting point is to make each observation’s
21 subject, scope, and authority explicit.

22 The architecture separates semantic state, which records source-linked facts and unresolved obliga-
23 tions, from operational state, which records work and decisions, and artifact storage, which retains
24 candidate bytes and evidence. A proposed transition identifies evidence for its declared acceptance
25 conditions. Unknown dependencies and unsupported translations remain visible obligations. Neither
26 persistence nor a cryptographic commitment turns an observation into proof of an unstated property.

27 The contribution is a design for these evidence boundaries, conditional acceptance and reuse argu-
28 ments, and a bounded implementation comparison. The empirical question is whether adding native
29 source-linked semantic context to matched public raw context changes useful cold repair under a
30 fixed resource policy. The retained design contains eight repository families, two arms, and two
31 nested repetitions per arm. It tests the combined context mechanism, rather than every component of
32 the intended supervision loop. Symbolic routing, warm-cache reuse, publication efficacy, and native
33 proving performance require separate evidence.

34 A returned proposal is not a successful repair. Useful completion requires actual nonempty public and
35 hidden checks to pass within the measured resource rules; provider failures, timeouts, and incomplete

validation stay in the fixed denominator. This separation also governs the reporting of setup work, earlier pilot results, runtime interruptions, and unknown costs.

2 Method and trusted assumptions

Separate evidence classes. Semantic state links facts, dependencies, contracts, and unresolved obligations to source. Operational state records goals, tasks, attempts, leases, and decisions. Artifact storage preserves candidate bytes and accepted manifests. An evidence item binds a subject to a declared statement, checker, environment, and provenance. A passing test concerns the finite checks actually executed; a formal derivation concerns its encoded relation and premises; a grant concerns an actor’s permission to act now. These classes cannot substitute for one another. In particular, a signature binds a record to a key under cryptographic assumptions. It establishes neither checker adequacy nor the honesty of an observation.

Applicable evidence. The intended loop derives obligations from current source, chooses an admissible analysis or proposal route, checks an isolated candidate, and admits only evidence applicable to that candidate. Translation validation motivates checking a particular conversion against its stated relation [5]; a source-linked record alone is not such validation. Abstract facts likewise depend on an approximation relation [1]. A typed or content-addressed representation does not imply full source-language soundness.

Conditional acceptance and reuse. If publication succeeds only after checking the current parent, authorized scope, complete requirement manifest, admitted evidence, and expected-parent compare-and-swap, each successful successor satisfies those gates. Given a valid initial root and exclusive use of that publication operation, induction preserves the declared manifest policy. This does not establish enforcement by unexamined write paths or atomicity of external effects.

For test reuse, suppose the observation of test t under profile θ factors through a complete dependency projection:

$$\text{Obs}_\theta(t, s) = f_\theta(\pi_D(s)).$$

If f_θ is deterministic on every admitted state, equal projections imply equal observations. Completeness and determinism are substantive premises; hashing an incomplete dependency list, opaque fixture, or uncontrolled effect does not discharge them. Appendix A preserves the full conditional, unmachine-checked statements and the distinction between behavioral acceptance, action permission, and publication.

Executed scope. The final comparison exercises native context preparation, model proposals, candidate custody and AI inspection, and separate cold scoring. Earlier constructed controls and a local guard-before-write witness test named implementation boundaries; they do not establish that the complete intended loop ran as a live benchmark. The retained proof profile lacks the admitted backend, key provenance, and exact statement binding required for native proof admission. Neither native proving performance nor empirical reuse benefit is inferred from these arguments.

3 Frozen comparison

Population and treatment. The eight recruited families are Tomlkit, installer, Tornado, more-itertools, charset-normalizer, iniconfig, wheel, and Jinja. They are a convenience sample, not a representative sample of Python repositories. Two arms and two nested repetitions yield 32 fixed cells, 16 per arm. Repetition identifiers 104729 and 130363 identify design positions; they were not served to the model API as random seeds.

Before final outcomes, the A/B local-cold comparison was retained as an order-preserving subset of the original 192-cell plan. The remaining 160 factor cells were withdrawn, and eight additional target families were not recruited. C/D routing, warm caches, test reuse, publication efficacy, the original sixteen-family inferential objectives, and D/A promotion comparisons remain outside this experiment. The narrowed comparison is descriptive.

Arm A receives the selected public raw context. Arm B preserves the matched raw selection and adds actual native source-linked semantic context. Sixteen exact family/arm request commitments

precede final generation; each arm’s two repetitions use its fixed request. This treatment changes the combined request and cannot isolate semantic subcomponents or prompt compression. Public source and native preparation remain separate from confidential acceptance tests.

One proposal and independent cold execution. Each cell permits one Grok 4.6 proposal POST, requesting at most 4,096 output tokens through a strict structured edit response. There is no scientific fallback, retry, served seed, or reasoning-effort parameter. A failed request consumes its original cell rather than creating a replacement. The frozen order is executed sequentially.

Proposal execution uses one CPU, 2 GiB, and a 32-process limit. The nominal gateway and HTTP-wrapper bounds are 600 seconds; the child bound is 595 seconds and the socket timeout is 590 seconds. Compliance depends on actual measured intervals. A configured limit, missing measurement, or nonfinite value is not evidence that the budget was met.

The operator retains exact candidate bytes and a specific AI review before eligible scoring. That procedural review supplies no independent human annotation. Scoring runs separately in a cold, network-disabled container with one CPU, 2 GiB, and a 128-process limit, bounded by 120 host seconds and 115 child seconds. The scorer’s process limit was fixed prospectively for both arms and all eight families because unchanged public concurrency tests require up to 100 threads. The earlier 32-process public-baseline failure remains recorded; proposal and native-preparation limits remain 32.

Useful completion requires an admitted candidate, nonempty public and hidden checks all passing, and measured resource compliance. Failed, refused, expired, unscored, or incomplete cells receive no useful-completion credit. Unknown states remain explicit and prevent a complete-cohort analysis until their terminal accounting is resolved. An import or collection error with no hidden assertions executed is incomplete validation, not an observed hidden behavioral assertion failure. Separate scoring and sealed records do not establish comprehensive resistance to adversarial scorer manipulation.

Interruption and provenance. The original resource lease ended during cell 7. Its request, failure, cleanup, and unknown termination-sender evidence remain retained. A separately admitted managed lease and continuation runtime govern cells 8–32 without replaying the consumed prefix or changing requests, model, order, scorer, limits, or analysis. Both runtime source versions remain explicit. The lease is a cooperative allocation, not an exclusive physical resource reservation; per-phase process boundaries apply the configured resource limits.

Continuation checks add work around effect admission and receipt signing. The signed gateway interval ends before signing; whole operator invocation time extends through subprocess completion and output flush/synchronization. These distinct intervals must retain their endpoints rather than being presented as a single total host clock.

4 Analysis and evidence separation

For family f , arm a , and repetition r , let Y_{far} indicate admitted useful completion. Average the two fixed repetitions within each family and arm: $\bar{Y}_{fa} = (Y_{fa1} + Y_{fa2})/2$. The retained contrast is

$$\hat{\Delta} = \frac{1}{8} \sum_{f=1}^8 (\bar{Y}_{fB} - \bar{Y}_{fA}).$$

Families are the independent analysis units; 32 nested cells are not 32 independent repositories. A descriptive robust-family outcome additionally requires both repetitions to succeed and is not a promotion decision.

The frozen bootstrap draws eight whole family blocks with replacement, sharing each draw across arms and repetitions. It uses 20,000 resamples, Python 3.12 `random.Random` with seed 20260911, lexicographically ordered family identifiers, and exactly eight `randrange(8)` draws per resample. The 95% percentile endpoints use linear interpolation at $(B - 1)p$ for $p \in \{0.025, 0.975\}$ and $B = 20,000$. These intervals are conditional, descriptive summaries of eight convenience-selected families, not population-level coverage or superiority/noninferiority decisions.

Failures remain in the fixed outcome denominator. Resource summaries report their available-observation counts and missingness. Provider child, HTTP wrapper, gateway, scorer, and operator

132 clocks can overlap and are not added as disjoint costs. Shared native preparation is counted once.
133 Baseline checks, prior failed attempts, review, queue waits, cleanup, and artifact work retain their
134 own accounting scopes. Token usage and cost ticks are telemetry; unavailable settled charges and
135 unmeasured effort remain unknown.

136 The developmental pilot is a separate four-family, 24-cell A/B study with three nested repetitions. Its
137 retained labels include ten useful completions, five per arm, from 24 POSTs and 12 scores. Ten child
138 durations exceeded 595 seconds by less than 1 ms, and pilot cell 20 exceeded 600 seconds in both
139 wrapper and gateway clocks. These deviations and labels remain preserved; the final resource policy
140 is not applied retrospectively to relabel the pilot.

141 Earlier boundary evidence is also separate from final repair outcomes. The mutation/cold comparison
142 retains one false reuse among all 26 cases, rather than reporting a selected zero-of-25 subset. All four
143 later correction invocations, including failed and partial attempts, remain visible; corrected full-cold
144 execution supplies no positive reuse credit. Constructed gate checks and public-baseline passes do
145 not count as useful final repairs.

146 5 Results

147 **Complete fixed-cohort accounting.** All 32 planned cells have verified terminal records; none is
148 missing or replaced. The experiment made 32 original proposal POSTs and 16 scoring invocations,
149 of which 15 completed cold validation. Useful completion was 9/32 (28.125%): A achieved 5/16
150 and B 4/16. Table 1 reports the family counts. Failures remain in these denominators. The mean of
151 the eight paired family differences, B minus A, is -0.0625 , or -6.25 percentage points. The frozen
152 20,000-resample whole-family bootstrap gives a descriptive 95% percentile interval of $[-0.1875, 0]$.
153 Each resample draws eight families and preserves both arms and their nested repetitions. The interval
154 describes this convenience sample; it supplies no population-level superiority, noninferiority, or
155 promotion decision.

156 **Concentration across families.** Tomlkit yielded one useful completion per arm, Tornado two for
157 A and one for B, and more-itertools two for each arm. The other five families yielded none. Seven
158 family differences are zero; the Tornado difference is -0.5 and accounts for the aggregate gap. Its B
159 cell 012 (repetition 130363) ended in a transport failure without a response or scorer invocation. This
160 outcome remains part of the fixed comparison. The data do not isolate an effect of semantic content,
161 a particular native component, or prompt compression. The treatment changes the combined request,
162 and the recorded endpoint includes operational and provider failures.

163 **Failure categories and incomplete validation.** The remaining 23 cells comprise 14 proposal-
164 child deadlines, two known proposal failures, and seven records with the original signed label
165 `hidden_acceptance_failed`. The two proposal failures were cell 003 (Tomlkit, A, 130363),
166 which returned HTTP 503, and the transport failure in cell 012. Their known local termination
167 does not establish whether a remote charge occurred. Among the seven hidden-failure labels, six
168 completed cold validation and failed executed hidden checks. Cell 005 (installer, A, 104729) instead
169 passed 28/28 public checks but collected zero hidden assertions and ended with an import-name
170 failure. It is incomplete hidden validation, neither an observed hidden behavioral assertion failure nor
171 a demonstrated external infrastructure fault. Its original label and zero useful credit are preserved; no
172 candidate, test, or failed request was revised and rerun.

173 **Measured time and unavailable costs.** The provider-child, HTTP-wrapper, and signed proposal-
174 gateway clocks each cover 32/32 cells. Their medians are 472.777, 473.660, and 474.107 seconds,
175 respectively. The 14 deadline cells are resource-unqualified under the frozen measured predicates;
176 configured caps do not imply that every measured duration complied. Scoring-gateway time is
177 available for 16/32 cells, with a median of 2.500 seconds across those invocations, including the
178 incomplete validation. Only one of eight families has complete scoring-time observations for
179 both repetitions of both arms. The frozen policy therefore leaves the paired scoring-cost interval
180 unavailable instead of imputing zeros or bootstrapping fewer families. Grant-to-terminal duration
181 includes waiting and bookkeeping: its median is 603.644 seconds and its maximum 10,938.574
182 seconds. These intervals overlap and are not summed. API usage is reported for 16/32 cells;

Table 1: Useful completions in the fixed 32-cell A/B cold comparison. Eight families are the analysis blocks; two repetitions per arm are nested.

Family	A useful / 2	B useful / 2	B – A mean
09-tomlkit	1	1	0
10-installer	0	0	0
11-tornado	2	1	-0.5
12-more-itertools	2	2	0
13-charset_normalizer	0	0	0
14-iniconfig	0	0	0
15-wheel	0	0	0
16-jinja	0	0	0
All eight (16 cells per arm)	5 / 16	4 / 16	-0.0625

183 settlement is unavailable for all 32. No dollar total, complete effort cost, or net efficiency gain is
184 established.

185 **Runtime and accounting deviations.** The original lease ended during cell 007. Its consumed
186 failure and unknown signal sender remain recorded; a prospectively admitted continuation covers
187 only cells 008–032, with separately identified executable sources. The original requests, order, limits,
188 and analysis policy remain fixed. The 26 additive disclosures also preserve pre-effect launch errors,
189 bookkeeping takeovers, diagnostic corrections, and interrupted operator work. Cell 027 had sufficient
190 credential headroom at its metadata check but only 895.290 seconds at the later reservation, below
191 the intended 900-second margin; exact POST time is unobserved. Cell 029’s delayed bookkeeping
192 enlarges its grant-to-terminal clock without creating another scientific trial. Outer invocation durations
193 and work after signed-gateway clock sampling retain separate scopes and missingness. They are not
194 added to the signed gateway interval or treated as measured AI reasoning effort.

195 **Evidence and comparison limits.** Fifteen failed proposals lacked original candidate-binding files.
196 Their later unsigned sidecars describe the unchanged public inventory and an empty patch; they are
197 derived metadata, not returned model edits, recovered original files, or successful candidates. The
198 four-family developmental pilot remains separate: its 24 cells yielded ten historical useful labels, five
199 per arm, with retained timing deviations rather than retrospective final-policy compliance. Earlier
200 boundary evidence in Table 2 is also separate from final repair outcomes. The ablation scope in
201 Table 3 records unmeasured or withdrawn endpoints. C/D routing, warm-cache reuse, publication
202 efficacy, and the original sixteen-family inferential objectives are not evaluated by these 32 cells.

203 6 Related work

204 Proof-carrying code places explicit evidence between a producer and a consumer’s safety policy [4].
205 This motivates separating an artifact from the evidence needed to accept it. Our finite-check repair
206 experiment and signed provenance records are not proof-carrying code in that formal sense. Abstract
207 interpretation and translation validation similarly motivate explicit approximation and conversion
208 relations [1, 5], not automatic admission of a backend or source-language proof.

209 Mokhov et al. separate scheduling from the decision whether a task needs rebuilding [2]. Our
210 reuse obligation additionally names the statement, environment, lifecycle, and trust conditions
211 for transferring a test observation. The cold comparison supplies no measured incremental-reuse
212 advantage over build systems.

213 SWE-agent studies an interface for repository navigation, editing, and program execution [6]; SWT-
214 Bench studies test-based validation of real-world fixes [3]. They motivate treating both information
215 access and executable checks as parts of a repair method. Differences in tasks, interaction, models,
216 and acceptance criteria preclude a head-to-head performance claim or an assertion that we reran their
217 benchmarks. The contribution here is a bounded composition of evidence and authority mechanisms,
218 not a first-of-kind claim or a new foundational algorithm.

219 7 Limitations

220 Eight convenience-selected families and two nested repetitions cannot support broad repository-
221 population inference. The intervention changes the complete context supplied to the model; com-
222 ponent ablations and a prompt-compression causal claim remain unavailable. The withdrawn C/D,
223 warm-cache, reuse, publication, and sixteen-family objectives are not completed outcomes.

224 Finite tests cover their executed assertions. Passing them does not establish complete behavioral cor-
225 rectness, source-level semantic fidelity, or adequate coverage of every relevant behavior. Independent
226 human semantic annotations, expert agreement, and human time were not measured. AI candidate
227 inspection does not supply those measurements.

228 The conditional design arguments retain checker adequacy, complete dependency projections, pro-
229 tected policy authority, and exclusive checked publication as premises. They establish no universal
230 Python theorem, native proof performance, global consensus, external exactly-once effects, or com-
231 plete adversarial scorer integrity. A local constructed witness cannot fill a missing production or
232 final-comparison measurement.

233 The interrupted runtime, actual timing deviations, prior failures, and unknown remote charges limit
234 net-efficiency claims. Lease occupancy is not compute usage. Overlapping clocks and a pilot cost
235 projection that aliases the same executions cannot be charged twice. Separate source epochs and
236 actual cap observations must remain visible even when a candidate passes its tests.

237 Numerical reproduction from anonymous, manifest-bound scalar derivatives is narrower than original-
238 signature verification or confidential-oracle rescoring. Changed derivative bytes are not authenticated
239 by the original signature. Reference source derivatives must be distinguished from executed source,
240 with upstream attribution and licences retained. Stable provenance hashes can permit correlation, so
241 a lexical anonymity audit is bounded and does not guarantee de-identification.

242 8 Conclusion

243 This study combines explicit evidence boundaries with a bounded cold-repair comparison. Across
244 eight historical families, the semantic-context arm produced 4/16 useful completions versus 5/16 for
245 matched raw context; the descriptive family-level interval provides no observed advantage for B. The
246 retained evidence distinguishes useful repairs, executed test failures, incomplete validation, provider
247 failures, and runtime deviations while preserving the fixed denominator. Conditional design arguments
248 remain subject to their trusted premises. Broader repository generalization, human semantic fidelity,
249 complete cost efficiency, and routing, reuse, or publication benefits require additional evidence
250 beyond this experiment.

References

- [1] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL '77, pages 238–252, Los Angeles, California, 1977. ACM Press. doi: 10.1145/512950.512973.
- [2] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP):1–29, 2018. doi: 10.1145/3236774. Expanded as Build systems à la carte: Theory and practice, *Journal of Functional Programming* 30, e11, 2020, doi:10.1017/S0956796820000088.
- [3] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. SWT-bench: Testing and validating real-world bug-fixes with code agents. In *Advances in Neural Information Processing Systems 37*, pages 81857–81887. Neural Information Processing Systems Foundation, 2024. doi: 10.52202/079017-2601.
- [4] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '97, pages 106–119, Paris, France, 1997. ACM Press. doi: 10.1145/263699.263712.
- [5] Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 1998)*, volume 1384 of *Lecture Notes in Computer Science*, pages 151–166. Springer, 1998. doi: 10.1007/BFb0054170.
- [6] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37*, pages 50528–50652. Neural Information Processing Systems Foundation, 2024. doi: 10.52202/079017-1601.

274 A Full conditional argument record

275 The following audited statements are retained verbatim from the earlier formal argument record.
 276 Their Equation 2, Algorithm 1, and statement identifiers name that record; they are not additional
 277 final experiments or numbered algorithms in the main text. References to a sealed profile or the
 278 absence of a campaign describe the scope of that argument audit. Later constructed corrections and
 279 the cold comparison do not retroactively discharge its universal premises. The intended algorithm's
 280 bounded retry branch is architectural specification; the final empirical protocol permits no proposal
 281 retry.

282 Conditional acceptance and reuse arguments (audited)

283 The statements below restate Equation 2, the accepted-lineage sketch, the dependency-projection
 284 reuse obligation, the three acceptance questions, and Algorithm 1 with explicit scope, premises,
 285 and conclusions. Every sketch is **unmachine-checked**: the sealed validation profile has no usable
 286 `z3`, `lean`, `coqc`, or `cvc5`, and no independently checked proof artifact discharges these arguments.
 287 Implementation evidence supports only the named tested premises. External side-effect atomicity
 288 and universal dependency closure are not inferred. Unknown specifications stay visible.

289 Statement FA-EQ2 (acceptance invariant)

290 **Scope.** One publication of an accepted lineage root under a declared artifact policy. The statement
 291 is a design invariant for that publication operation. It is not a theorem that every current write path
 292 enforces the invariant, and it is not a claim about Git, remote stores, or physical effects outside the
 293 tested expected-parent compare-and-swap.

294 **Premises.** Let s be the current accepted root, s' a candidate successor, a the requesting actor, and q
 295 the required obligation set. Write $\text{Publish}(s')$ for the unique accepted-lineage publication of s' .

Conclusion.

$$\begin{aligned} \text{Publish}(s') \Rightarrow & \text{CurrentParent}(s) \wedge \text{Authorized}(a, s) \\ & \wedge \text{Scoped}(a) \wedge \text{CompleteManifest}(q, s') \\ & \wedge \text{EvidenceAdmitted}(q, s') \wedge \text{CAS}(s, s'). \end{aligned} \quad (2')$$

296 **Argument (unmachine-checked).** If publication is defined to succeed only after those six gates,
 297 then any published s' satisfies them. The converse is not claimed: a candidate that meets the gates
 298 may still lose compare-and-swap. Failed, stale, incomplete, or simulated attempts are not Publish
 299 successors.

300 **What the invariant does not establish.** Properties omitted by the checked policy; truth of unsup-
 301 ported or lossy translations; honesty of signed observations; atomicity of external side effects; global
 302 consensus; or universal enforcement by unexamined write paths.

303 Statement FA-I31 (accepted-lineage induction)

304 **Scope.** Roots reachable *solely* through accepted publication transitions of the operation in FA-EQ2,
 305 under a fixed statement profile Θ and policy predicate P_Θ over artifact manifests.

306 **Premises.**

- 307 1. **Initial-root policy.** The initial accepted root satisfies P_Θ .
- 308 2. **Exact input.** Required subject bytes and dependency references are verified before admis-
 309 sion.
- 310 3. **Complete manifest.** The requirement manifest is complete under an admitted completeness
 311 rule; membership of some units is not completeness.
- 312 4. **Checker.** Each accepted evidence item satisfies its designated checker relation and evidence
 313 class.

- 314 5. **Translation bound.** Lossy or unsupported translations cannot discharge source-level
 315 obligations.
- 316 6. **Current parent.** Publication names the expected current parent.
- 317 7. **Exclusive publication.** Compare-and-swap linearizes concurrent same-parent writers; losers
 318 are not successors.
- 319 8. **Protected authority.** Policy and checker identities cannot change under ordinary worker
 320 authority.

321 **Conclusion.** Every root reachable solely through accepted publication transitions satisfies P_Θ .

322 **Argument (unmachine-checked).** Induction on the number of successful publications. The base
 323 case is premise 1. For the inductive step, the publisher checks P_Θ on the exact candidate manifest
 324 and current parent before compare-and-swap linearization. Failed, stale, incomplete, or simulated
 325 attempts are excluded from the successor relation, so they are not inductive cases. The induction
 326 preserves only properties actually expressed by P_Θ .

327 **Nonvacuity.** The successor relation is nonempty on the tested sealer: a complete current manifest
 328 can publish, while an empty or incomplete mandatory manifest cannot. Deny-all publication would
 329 make the invariant hold vacuously and is not a useful-progress witness.

330 **Statement FA-I32 (dependency-projected test reuse)**

331 **Scope.** Transfer of a previous setup/call/teardown observation of test t between states s and s' under
 332 a declared runtime/effect profile θ . The statement is a proof *obligation*, not a theorem of arbitrary
 333 pytest execution.

334 **Premises.** Let $\text{Obs}_\theta(t, s)$ be the relevant observation. Suppose there exist a complete projection
 335 π_D and a deterministic function f_θ such that

$$\text{Obs}_\theta(t, s) = f_\theta(\pi_D(s))$$

336 on every state admitted by the reuse policy.

337 **Conclusion.** If $\pi_D(s) = \pi_D(s')$, then $\text{Obs}_\theta(t, s) = \text{Obs}_\theta(t, s')$. An admitted previous observation
 338 may transfer only inside that profile and those assumptions.

339 **Argument (unmachine-checked).** Immediate from functionality of f_θ . The difficult premises
 340 are completeness and determinism of π_D . Equal digests of incomplete manifests do not satisfy π_D
 341 completeness. Opaque, uncontrolled, or unknown fixture values, incomplete runtime traces, and
 342 teardown failures do not discharge the obligation; they require execution or fail closed. A cold
 343 full-run oracle and adversarial mutations can check an implementation; they do not prove universal
 344 dependency closure. No such independent cold-oracle campaign is recorded here.

345 **Statement FA-I33 (three acceptance questions)**

346 **Scope.** Any candidate action that might be described as “accepted.”

347 **Premises and conclusions (separate).**

Question	What a positive answer establishes, and what it does not
Behavioral claim	The candidate program or model satisfies its stated obligation under the declared checker, environment, and evidence class. This is not permission to act and not publication of a lineage root.
348 Action permission	This actor may perform this action now under the bound lease, policy, scope, and roots. This is not behavioral correctness and not publication.
Publication	This candidate and its evidence become the current accepted root. This is not, by itself, proof that omitted obligations hold or that external effects occurred exactly once.

349 Neither a proof of behavior nor a capability grant answers all three. Formalization fixes the statement;
 350 authorization binds actor, tool, arguments, resources, effects, scope, time, and roots; execution
 351 observations report what happened; publication chooses the accepted lineage.

352 **Statement FA-ALG1 (composed supervision loop)**

353 **Scope.** Algorithm 1 remains the intended composition: a control-flow specification, not a newly
 354 executed full-loop result. It is labeled **executed composition** only on the steps supported by the
 355 integrated guard-before-write witness. Passing isolated stop-path tests does not establish the complete
 356 loop.

357 **Intended algorithm.** Input: authorized query q , current source forest F , policy P , budget B .

- 358 1. Reconcile event, task, and artifact state; refresh F and derive G, O .
- 359 2. If cancelled or budget exhausted: return explicit non-success.
- 360 3. If every current obligation is admitted: return goal-completion verification.
- 361 4. Generate bounded task proposals; preview and separately admit ready work.
- 362 5. If no task is admissible: return wait-or-review, not goal success.
- 363 6. Claim the task; bind lease/fence, preimages, required context, and validation.
- 364 7. Choose a deterministic, authorized residual-model, review, or unavailable route.
- 365 8. If review or unavailable: record a blocked attempt; return to the event service.
- 366 9. Produce and apply a candidate only through checked isolated execution.
- 367 10. Rescan; derive O_{new} from authorized rules, not the worker's pass claim.
- 368 11. Verify or reuse evidence; retain failed, unknown, and pending obligations.
- 369 12. If the failure is refinable: record a new residual; return a bounded retry.
- 370 13. If evidence is incomplete or unsafe: return defer-or-reject.
- 371 14. Persist the manifest; recheck current bindings; compare-and-swap the expected parent.
- 372 15. If publication conflicts: return reconcile, not completion.
- 373 16. Record the accepted task transition and updated F, G, O ; reevaluate at line 1.
- 374 17. Propose procedure or policy learning separately; never self-promote.

375 **Executed composition (integrated witness only).** The integrated witness supports a frozen guard-
 376 before-write cycle: authorized pre-state and obligation; counterexample; admitted deterministic route;
 377 isolated patch application; independent tests; narrow cache reuse of an unchanged eligible receipt;
 378 complete-manifest publication; and rejection of invalid, incomplete, and simulated non-completion.
 379 Those steps correspond to intended lines 4, 7 (deterministic), 9, 11 (narrow reuse), 13, 14, and 16,
 380 plus the negative reading of line 17 (no worker self-promotion). They do *not* execute the live event-
 381 service loop, residual production-model dispatch, budget-exhaustion composition, wait-or-review,
 382 procedure/policy learning, native proof, federation, or a matched A–D experiment.

383 **Trusted limitations retained with the algorithm.** Unknown or inconsistent specifications remain
 384 visible. Incomplete projections cannot silently authorize reuse. External exactly-once behavior
 385 outside the tested write-ahead-log / compare-and-swap / lifecycle boundary is not claimed. Optional
 386 native proving and world/procedure extensions are out of scope unless a later task supplies an eligible
 387 profile and consumed-result evidence.

388 **B Family-level display**

Figure 1 displays the same family counts as Table 1; it is not an additional experiment.

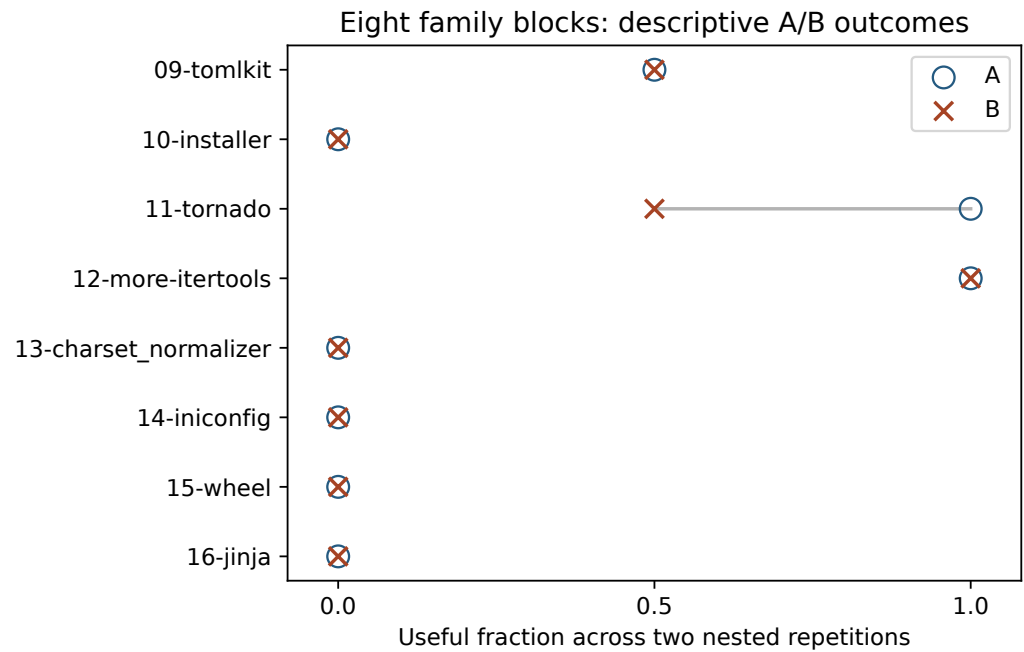


Figure 1: Useful completions by family and arm in the same 32-cell comparison. Each arm has two nested repetitions per family; the figure adds no trials.

389

C Historical boundary evidence

Table 2 retains the historical qualification scope separately from the final repair comparison. Identifiers of the form NS-NNN name internal qualification records, not repository families or independent repair trials. The anonymous boundary component retains the detailed evidence index and limitations.

Table 2: Historical evidence boundaries. Local controls, unavailable measurements, and the retained false-reuse case are not additional final repair outcomes.

Boundary	Safe rejection	Valid progress	Evidence
Provider gate	Missing/mismatched authority and untrusted close rejected in fixtures.	Deterministic repair and injected residual fixture; no production routing claim.	NS-008
Loss-aware translation	Modal erasure and unvalidated solver lift refused.	Exact/conservative translation only; no kernel source proof.	NS-009
Native theorem/circuit	Untested: no admitted native proof profile.	Unavailable: no genuine proof or measured prove/verify run.	NS-014 scope closure
Fixture/call/teardown reuse	Prior false reuse 1/26 retained; corrected route requires full cold.	Full public cold baseline only; no admitted positive reuse or speedup.	NS-011; corrected NS-027
Publication after restart	Empty/stale/failed publication refused in local controls.	WAL/CAS recovery and valid continuation; explicit hermetic shims.	NS-012/013
World/procedure consumption	Untested/out of scope.	Untested: no retained paired pre/post consumption.	NS-015 scope closure

D Withdrawn and unmeasured comparisons

Table 3: Withdrawn and unmeasured comparisons. These entries are scope dispositions, not zero-effect measurements.

Endpoint	Scope
C/D routing/governance comparative benefits	Withdrawn or unmeasured; no final comparison or effect estimate.
proof/test reuse or local_warm effects	Withdrawn or unmeasured; no final comparison or effect estimate.
certificate/publication comparative benefits	Withdrawn or unmeasured; no final comparison or effect estimate.
adversarial scorer-integrity qualification	Withdrawn or unmeasured; no final comparison or effect estimate.
independent human semantic annotation	Withdrawn or unmeasured; no final comparison or effect estimate.
Non-A/B planned cells (128)	Withdrawn; no measured ablation rows.
A/B warm planned cells (32)	Withdrawn; no measured ablation rows.

E LLM use, public reproduction, and responsible use

Scientific model use. The final cold-repair comparison uses Grok 4.6 to propose source edits from the fixed A/B context requests (Section 3). Each of the 32 cells consumes one proposal POST with a maximum of 4,096 requested output tokens and a structured edit response. No scientific fallback, retry, served random seed, or reasoning-effort parameter is used. The model’s weights and exact served revision are unavailable. Candidate inspection by AI is a procedural control; it is not independent human assessment or a substitute for executed checks.

Research and manuscript assistance. AI assistance also contributed to source recovery and drafting, implementation and debugging, experiment preparation and candidate inspection, and analysis and artifact preparation. The authoring supervisors were configured to use Grok 4.6, with Codex gpt-5.6-terra at high reasoning effort as a fallback only following independently verified primary-quota exhaustion. This describes the configured authoring policy, not a claim that every task used a fallback or that these models generated the scientific comparison under a mixed route. The configured policy does not identify the model behind every separate interactive assistance session. AI-assisted checking supplies no measured human semantic fidelity, expert agreement, or human effort time, and does not discharge the conditional arguments in Appendix A.

Anonymous numerical reproduction. The accompanying supplement keeps the 32-cell final comparison, the separate 24-cell developmental pilot, and historical boundary controls in distinct components. Its top-level README.md specifies Python 3.12 and the command `python3 -B reproduce.py -root ROOT -output OUTPUT`, where ROOT is a fresh extraction and OUTPUT is a fresh directory outside it. The program checks the manifest-bound files and reproduces their retained numerical summaries; it does not generate new proposals, execute candidates or hidden tests, rerun historical qualifications, or authenticate original signed records. Hidden tests, reference patches, credentials, keys, and raw private operational logs are excluded. Original and derivative hashes have distinct roles: original signatures do not authenticate modified anonymous derivative bytes. Exact executed source and labeled reference derivatives remain distinct. Upstream retrieval metadata, revisions, and retained license and copyright notices accompany the released assets. Stable hashes and timestamps may permit correlation, so these release controls do not guarantee anonymity.

Potential impacts and limits. Explicit evidence boundaries may help users identify stale checks and unsupported acceptance claims, but this experiment establishes no general reliability or efficiency benefit. Incorrect repairs, incomplete tests, or mistaking signed provenance for correctness could lead to unsafe deployment; automated coding can also be misused to introduce harmful changes. The stated scope, isolated checks, restricted artifact projection, and visible failures reduce particular risks without establishing comprehensive adversarial resistance. No model weights or human-subject dataset are released, and no human semantic annotations, expert-agreement study, or human-time measurements were collected for the reported comparison.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and Sections 1, 5, and 7 restrict the contribution to explicit evidence boundaries, conditional arguments, and the fixed eight-family A/B comparison (A 5/16, B 4/16). They do not claim semantic correctness, a general performance advantage, or benefits from withdrawn routing, reuse, or publication comparisons.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Section 7 states the convenience-sample scope, combined-context intervention, finite-test and trusted-premise limitations, and missing human semantic fidelity, expert agreement, and human-time measurements. Sections 4 and 5 retain failures, timing deviations, overlapping clocks, incomplete validation, and unsettled charges.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [No]

Justification: Section 2 and Appendix A give explicit scopes, premises, and conditional argument sketches, but do not claim a complete formal proof of the intended implementation. The arguments are unmachine-checked, and dependency completeness, checker adequacy, and exclusive checked publication remain assumptions.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [No]

Justification: Sections 3 and 4 disclose the retained protocol and frozen reduction, and Section E describes the anonymous numerical replay. Independent regeneration and confidential-oracle rescoring are not supplied: exact served model revisions, private original custody, and hidden tests are unavailable through that replay.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The anonymous supplement provides manifest-bound inputs, selected source versions, retained license/retrieval metadata, and Python 3.12 instructions that reproduce the reported scalar summaries. It does not provide model weights, hidden-oracle access, or public authentication of original private signatures; Section E states these limits.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 3 specifies the eight families, matched A/B context, two nested repetitions, one Grok 4.6 POST per cell, response cap, no scientific retry or fallback, separate cold scoring, and measured resource predicates. There is no model training or tuning in this comparison, and Sections 4 and 5 specify the analysis and retain the withdrawn design factors and deviations.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Sections 4 and 5 define the paired family contrast and the 20,000-resample whole-family percentile interval, including seed, ordering, interpolation, and the eight-family block unit. The interval $[-0.1875, 0]$ is explicitly conditional and descriptive; missing paired scoring-cost observations are not imputed or used to claim population coverage, superiority, equivalence, or noninferiority.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: Sections 3, 4, and 5 report per-phase CPU, memory, process and time limits, available measured clocks, prior work, and missingness. Provider-side hardware and total compute/effort or settled charges are unavailable, and overlapping clocks, reservation lifetimes, and configured caps are not presented as measured total compute.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [N/A]

Justification: Author review of and conformance with the NeurIPS Code of Ethics has not been attested in the preparation record. This answer records unavailable author confirmation, not a claim that the work violates the code; the author must review it before submission.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section E discusses potential benefits of making evidence boundaries explicit, and harms from incorrect repairs, misplaced trust in signed provenance, unsafe deployment, malicious edits, and residual privacy correlation. It states the limited scope of the proposed mitigations and makes no measured societal-benefit or comprehensive security claim.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The release contains no model weights or human-subject dataset of the high-risk types described by this question. Section E and the supplement documentation nevertheless describe excluded private logs, credentials, keys, hidden tests and reference patches, and the residual limits of anonymization.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.

- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No]

Justification: The supplement retains upstream revisions, retrieval metadata, license and copyright notices, and distinguishes exact executed source from reference derivatives. A complete confirmation of all provider and asset terms of use is unavailable in this preparation, so retained notices alone are not asserted to establish full compliance.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The supplement documents its final-comparison, developmental-pilot, and historical-boundary components with separate manifests, numerical reproduction commands, source and derivative identities, and explicit data-access and inference limitations. Section E explains this scope, including the distinction between integrity hashes and authentication of private originals.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

749 Answer: [N/A]

750 Justification: The reported source-code comparison includes no crowdsourcing or human-

751 subject annotation study. Section 7 states that human semantic fidelity, expert agreement and

752 human time were not measured; AI candidate inspection is not relabeled as human research.

753 Guidelines:

- 754 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
- 755 with human subjects.
- 756 • Including this information in the supplemental material is fine, but if the main contribu-
- 757 tion of the paper involves human subjects, then as much detail as possible should be
- 758 included in the main paper.
- 759 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,
- 760 or other labor should be paid at least the minimum wage in the country of the data
- 761 collector.

762 **15. Institutional review board (IRB) approvals or equivalent for research with human**

763 **subjects**

764 Question: Does the paper describe potential risks incurred by study participants, whether

765 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)

766 approvals (or an equivalent approval/review based on the requirements of your country or

767 institution) were obtained?

768 Answer: [N/A]

769 Justification: No human-subject experiment or crowdsourcing is reported, so no participant-

770 risk, compensation or IRB-approval result is claimed. The absence of human semantic

771 annotations and expert-agreement measurements is explicit in Section 7; this answer does

772 not invent institutional approval.

773 Guidelines:

- 774 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
- 775 with human subjects.
- 776 • Depending on the country in which research is conducted, IRB approval (or equivalent)
- 777 may be required for any human subjects research. If you obtained IRB approval, you
- 778 should clearly state this in the paper.
- 779 • We recognize that the procedures for this may vary significantly between institutions
- 780 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
- 781 guidelines for their institution.
- 782 • For initial submissions, do not include any information that would break anonymity (if
- 783 applicable), such as the institution conducting the review.

784 **16. Declaration of LLM usage**

785 Question: Does the paper describe the usage of LLMs if it is an important, original, or

786 non-standard component of the core methods in this research? Note that if the LLM is used

787 only for writing, editing, or formatting purposes and does *not* impact the core methodology,

788 scientific rigor, or originality of the research, declaration is not required.

789 Answer: [Yes]

790 Justification: Section 3 and Section E identify methodology-essential Grok 4.6 scientific

791 proposals under one-POST/no-fallback rules and describe AI assistance in implementation,

792 inspection, analysis and writing. The configured authoring Grok-to-Codex gpt-5.6-terra

793 high quota-only fallback is distinguished from observed scientific use, without claiming that

794 every fallback or interactive session used that model.

795 Guidelines:

- 796 • The answer [N/A] means that the core method development in this research does not
- 797 involve LLMs as any important, original, or non-standard components.
- 798 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
- 799 be described.